PROJECT

10

DEPLOY AN APP ACROSS ACCOUNTS

DOCKER SERVICE

Prepared by:
Amrize JK
Certified Solutions Architect Associate

Objective

This project aims to containerize a web application using Docker by creating and pushing a custom image to Amazon ECR.

It then demonstrates how two AWS accounts can securely access and run each other's container images using IAM roles and AWS CLI authentication.

Services

Index.html - The main frontend file that displays the web content to users.

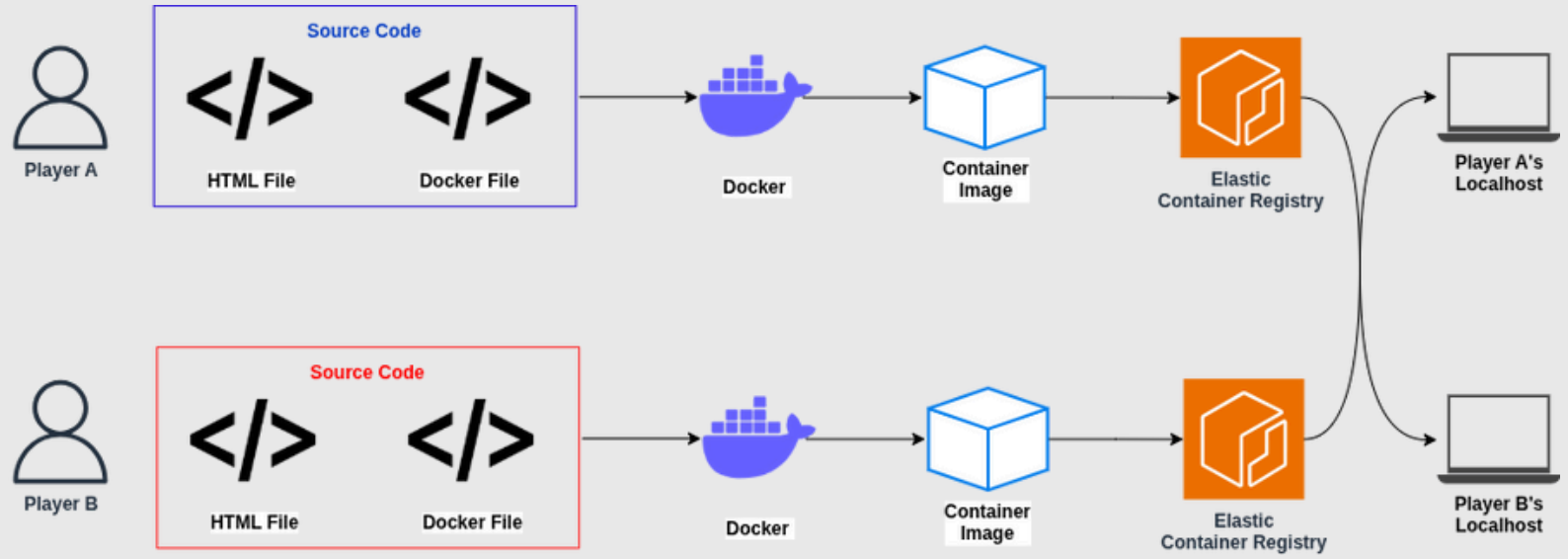
Docker - Used to containerize the application by building a custom image using a Dockerfile.

Amazon ECR - Stores the Docker images for deployment and sharing across AWS accounts.

AWS CLI - Enables command-line access to AWS services for login, image push/pull, and configuration.

AWS IAM - Manages cross-account access permissions using roles and policies.

Architecture Diagram



The web app is containerized into a Docker image and pushed to Amazon ECR by Player A. Player B, from a different AWS account, authenticates via AWS CLI and pulls the image using granted IAM permissions. Both users can run and deploy each other's images, enabling secure, cross-account container sharing and deployment.

Steps to Implement

1. Install Docker on your system:
Download and install Docker from the official Docker website. Verify the installation by running docker --version in your terminal.

2. Log in to AWS with an IAM user:
Create or use an IAM user with ECR and CLI access in both AWS accounts. Configure the AWS CLI using aws configure by entering the access key, secret key, region, and output format.

3. Set up Amazon ECR in Account A:
In Account A, create a new ECR repository using the AWS Management Console or CLI. This will store the Docker image for sharing.

4. Build and push the Docker image:
Write a Dockerfile to containerize the app (e.g., index.html). Build the image using docker build -t app-name ., then authenticate and push it to ECR using docker push.

5. Configure cross-account access:
Update the ECR repository policy in Account A to allow Account B's IAM user or role to pull images by adding a cross-account permissions policy.

6. Pull and run the image from Account B:
In Account B, authenticate using the AWS CLI, then pull the image from Account A's ECR using docker pull. Finally, run the container using docker run -p 80:80 app-name.

Story Time

Docker (The Magic Box Maker):
Docker is your enchanted toolbox that packs your app code, files, and spells into a neat little magic box (container) that works the same in every kingdom (system).

IAM User (The Gatekeeper):
The IAM user is the royal gatekeeper of each AWS kingdom. They hold the secret keys (access keys) to enter the cloud castle and use tools like ECR and CLI.

ECR (The Royal Container Vault):
Amazon ECR is the golden vault where your magic boxes (container images) are stored. Player A places their enchanted image here, allowing Player B to access it if granted permission.

AWS CLI (The Spellcaster's Staff):
The AWS CLI is your command-line interface that lets you cast cloud spells. It helps you log in, push/pull container boxes, and talk to the cloud kingdom directly.

Cross-Account Access (The Magic Bridge):
This enchanted bridge lets friends from other kingdoms (AWS accounts) access your container vault. With the right IAM spells (policies), Player B can cross into Player A's vault.

Container Run (Summoning the Magic):
Once Player B pulls the box with docker pull, they summon it using docker run. Instantly, the magic springs to life running the app just like in the original kingdom.

BY
AMRIZE J.K

THANK YOU!

nextwork